



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Analisis y Diseño de Algoritmos

Practica 3

Gonzalez Ramirez Ángeles Nataly

Melo Cruz Leonardo Galdino

Olvera Gil Leonel

3BV1

https://github.com/leonardogmc/Practica3-DaC

1. Introducción

El presente reporte analiza los resultados obtenidos en la Práctica 3 de Diseño y Análisis de Algoritmos, en la cual se estudia el comportamiento de algoritmos de divide y vencerás mediante dos problemas: la selección del k-ésimo elemento con Quickselect y la multiplicación de matrices mediante el método tradicional y el algoritmo de Strassen.

Las pruebas permiten comparar no solamente la complejidad teórica de los algoritmos, sino también su comportamiento real al ser implementados en dos lenguajes diferentes: C++ y Python. Se registraron tiempos de ejecución y consumo de memoria para distintos tamaños de entrada.

En Quickselect se utilizaron arreglos de enteros de 32 y 64 bits. En la multiplicación de matrices se evaluaron representaciones bidimensionales (2D) y lineales (1D), comparando el método tradicional con Strassen. El código utilizado corresponde al repositorio de la práctica, que contiene las carpetas 01_Quickselect y 02_Strassen.

2. Objetivos

- Evaluar el rendimiento de Quickselect para diferentes tamaños de entrada.
- Comparar el consumo de memoria utilizando enteros de 32 y 64 bits.
- Comparar la multiplicación tradicional de matrices con Strassen.
- Analizar el efecto de las representaciones 1D y 2D.
- Comparar las implementaciones equivalentes de C++ y Python.
- Explicar las causas de las diferencias de tiempo observadas.

3. Fundamento de los algoritmos

3.1 Quickselect

Quickselect utiliza una partición alrededor de un pivote para localizar el elemento que ocupa la posición k sin ordenar completamente el arreglo. Después de la partición, solamente se continúa con la parte que contiene la posición buscada. Su complejidad esperada es $O(n)$, aunque el peor caso puede alcanzar $O(n^2)$, dependiendo de la elección de los pivotes.

3.2 Multiplicación tradicional

La multiplicación tradicional calcula cada elemento de la matriz resultado mediante tres niveles de iteración. Para matrices cuadradas de tamaño n, su complejidad temporal es $O(n^3)$.

3.3 Strassen

Strassen divide las matrices en submatrices y reduce el número de multiplicaciones recursivas. Su complejidad es aproximadamente $O(n^{2.807})$, inferior a $O(n^3)$. Sin embargo, realiza más sumas y restas y necesita matrices auxiliares, por lo que el beneficio puede depender del tamaño de la matriz y de la forma en que se almacenan los datos.

4. Metodología

Las pruebas se realizaron con los mismos tamaños de entrada en C++ y Python. La versión de C++ mostrada en las evidencias fue compilada con C++17 y la opción de optimización -O3.

Para Quickselect se utilizaron n = 1,000; 5,000; 10,000; 50,000; 100,000; 500,000; 1,000,000; 5,000,000 y 10,000,000, con k = n/2. Se midieron tiempo y memoria para 32 y 64 bits.

Para matrices se utilizaron n = 16, 32, 64, 128, 256 y 512. Se compararon cuatro variantes: Tradicional 2D, Tradicional 1D, Strassen 2D y Strassen 1D.

5. Resultados de Quickselect

n	k	C++ 32 (s)	C++ 32 (MB)	C++ 64 (s)	C++ 64 (MB)	Python 32 (s)	Python 32 (MB)	Python 64 (s)	Python 64 (MB)
1,000	500	0.000007	0.0038	0.000008	0.0076	0.000432	0.0038	0.000320	0.0076
5,000	2,500	0.000026	0.0191	0.000026	0.0381	0.001919	0.0191	0.001766	0.0381
10,000	5,000	0.000060	0.0381	0.000061	0.0763	0.004947	0.0381	0.004111	0.0763
50,000	25,000	0.000243	0.1907	0.000251	0.3815	0.014967	0.1387	0.004216	0.3715
100,000	50,000	0.000622	0.3815	0.000623	0.7629	0.027753	0.3815	0.029502	0.7629
500,000	250,000	0.002466	1.9073	0.002479	3.8147	0.189969	1.9073	0.199519	3.8147
1,000,000	500,000	0.006284	3.8147	0.006561	7.6294	0.325596	3.8147	0.350676	7.6294
5,000,000	2,500,000	0.027572	19.0735	0.028619	38.1470	1.571735	19.0735	1.623334	38.1470
10,000,000	5,000,000	0.056588	38.1470	0.059252	76.2939	2.154476	38.1470	2.231174	76.2939

5.1 Análisis de Quickselect

Los resultados muestran una ventaja clara de C++ en tiempo de ejecución. Para n = 10,000,000, C++ tarda 0.056588 s con 32 bits, mientras que Python tarda 2.154476 s. En 64 bits, C++ tarda 0.059252 s frente a 2.231174 s de Python. Por lo tanto, Python tarda aproximadamente 38 veces más en la prueba de 32 bits y 38 veces más en la de 64 bits.

En memoria, los resultados muestran aproximadamente 38.147 MB para 32 bits y 76.294 MB para 64 bits con 10 millones de elementos. Esto coincide con el hecho de que un entero de 64 bits requiere aproximadamente el doble de bytes que uno de 32 bits.

6. Resultados de multiplicación de matrices

Resultados de C++:

n	Trad. 2D (s)	Mem (MB)	Trad. 1D (s)	Mem (MB)	Strassen 2D (s)	Mem (MB)	Strassen 1D (s)	Mem (MB)
16	0.000005	0.002	0.000002	0.002	0.000003	0.002	0.000002	0.002
32	0.000017	0.009	0.000013	0.008	0.000057	0.053	0.000030	0.045
64	0.000123	0.033	0.000100	0.031	0.000402	0.195	0.000134	0.180
128	0.001058	0.128	0.001003	0.125	0.003073	0.750	0.001349	0.719
256	0.009525	0.506	0.021452	0.500	0.019447	2.937	0.008084	2.875
512	0.087716	2.012	0.503919	2.000	0.126018	11.623	0.044857	11.500

Resultados de Python:

n	Trad. 2D (s)	Mem (MB)	Trad. 1D (s)	Mem (MB)	Strassen 2D (s)	Mem (MB)	Strassen 1D (s)	Mem (MB)
16	0.009712	0.002	0.009794	0.002	0.009723	0.002	0.009883	0.002
32	0.076016	0.008	0.075713	0.008	0.068789	0.032	0.068839	0.032
64	0.614079	0.031	0.618150	0.031	0.490335	0.120	0.487254	0.120
128	5.074514	0.125	5.994543	0.125	3.581939	0.471	3.567903	0.471
256	40.919174	0.500	53.153677	0.500	35.694036	1.877	40.006469	1.877
512	327.353482	2.012	432.128743	2.000	255.483921	7.508	284.152893	7.480

6.1 Análisis de las matrices

En C++, para n = 512, la multiplicación tradicional 2D tarda 0.087716 s y Strassen 2D tarda 0.126018 s. En este caso, el método tradicional es más rápido. Esto se explica por los costos adicionales de Strassen: división recursiva, sumas y restas y creación de matrices auxiliares.

La representación 1D cambia notablemente el resultado. Para n = 512 en C++, Strassen 1D tarda 0.044857 s frente a 0.503919 s del tradicional 1D, por lo que en esta prueba Strassen 1D es aproximadamente 11.2 veces más rápido que el tradicional 1D.

En memoria, Strassen necesita más espacio. Para n = 512 en C++, Strassen 2D utiliza 11.623 MB frente a 2.012 MB del tradicional 2D. En Python se registran 7.508 MB y 2.012 MB, respectivamente, según la métrica de memoria utilizada en los programas.

7. Comparación de las versiones C++ y Python

Aspecto	C++	Python
Ejecución	Código compilado a código nativo.	Ejecución mediante el entorno de Python.
Quickselect	Tiempos muy bajos en todos los tamaños.	Mucho más lento al aumentar n.
Matrices	Tiempos especialmente bajos.	Los tiempos crecen drásticamente.
Operaciones numéricas	Tipos primitivos y acceso de bajo nivel.	Mayor sobrecosto por operación.
Optimización	La compilación con -O3 permite optimizaciones.	No obtiene la misma optimización nativa del compilador.
Memoria	Control más directo de tipos y estructuras.	Mayor sobrecosto interno en estructuras generales.

8. Explicación del problema de Python en el tiempo de ejecución

El problema observado en Python no corresponde a un error del algoritmo. La diferencia se debe principalmente al costo de ejecutar una gran cantidad de operaciones numéricas desde código Python. En Quickselect se realizan numerosas comparaciones, accesos, asignaciones y operaciones de partición. En la multiplicación de matrices, el número de operaciones crece todavía más debido a la complejidad $O(n^3)$ del método tradicional.

C++ se compila a código nativo y el compilador puede aplicar optimizaciones como eliminación de operaciones innecesarias, optimización de bucles y mejor aprovechamiento de la memoria. En la ejecución mostrada, además, se utilizó -O3. Python introduce un costo adicional por operación debido a su modelo de ejecución, tipado dinámico y representación general de objetos.

La diferencia se amplifica al crecer el tamaño de entrada. Por ejemplo, para matrices de 512 × 512, la versión tradicional 2D tarda 0.087716 s en C++ frente a 327.353482 s en Python. Esto representa aproximadamente 3,731 veces más tiempo para Python. En Strassen 2D, C++ tarda 0.126018 s y Python 255.483921 s, una diferencia de aproximadamente 2,027 veces.

Por tanto, aunque ambos lenguajes implementan el mismo algoritmo y presentan la misma complejidad asintótica, el tiempo absoluto puede ser muy diferente. La complejidad Big-O describe principalmente cómo crece el trabajo con respecto al tamaño de entrada, pero no elimina los costos constantes ni las diferencias producidas por el lenguaje, el compilador, la representación de datos y el acceso a memoria.

También es importante señalar que Python puede alcanzar un rendimiento mucho mejor para este tipo de problemas cuando las operaciones pesadas se delegan a bibliotecas implementadas en C/C++ o Fortran, como ocurre con algunas operaciones vectorizadas de NumPy. Sin embargo, las mediciones de esta práctica corresponden a las implementaciones directas utilizadas en los programas, por lo que no deben interpretarse como una comparación de todo el ecosistema de Python contra C++.

9. Principales resultados

- C++ presenta una ventaja muy grande en tiempo de ejecución frente a Python en las pruebas realizadas.
- Para 10 millones de elementos, Quickselect tarda aproximadamente 38 veces más en Python.
- El cambio de 32 a 64 bits aproximadamente duplica la memoria requerida por los datos de Quickselect.
- Strassen tiene mejor complejidad asintótica que el método tradicional, pero sus sobrecostos pueden hacerlo más lento en determinados tamaños.
- La representación 1D puede mejorar notablemente el rendimiento; en C++ con n=512, Strassen 1D fue la variante más rápida.
- Strassen utiliza más memoria por las matrices auxiliares necesarias durante la recursividad.
- El crecimiento $O(n^3)$ de la multiplicación tradicional amplifica las diferencias de rendimiento entre C++ y Python.

10. Capturas

```
C:\Users\meloc\OneDrive\Desktop>g++ -std=c++17 -O3 Pract3.cpp -o Pract3

C:\Users\meloc\OneDrive\Desktop>.\Pract3
================================================================================
QUICKSELECT - TIEMPO Y CONSUMO DE MEMORIA
================================================================================
           n           k      Tiempo 32(s)    Memoria 32(MB)      Tiempo 64(s)    Memoria 64(MB)
--------------------------------------------------------------------------------
        1000         500          0.000007            0.0038          0.000008            0.0076
        5000        2500          0.000026            0.0191          0.000026            0.0381
       10000        5000          0.000060            0.0381          0.000061            0.0763
       50000       25000          0.000243            0.1907          0.000251            0.3815
      100000       50000          0.000622            0.3815          0.000623            0.7629
      500000      250000          0.002466            1.9073          0.002479            3.8147
     1000000      500000          0.006284            3.8147          0.006561            7.6294
     5000000     2500000          0.027572           19.0735          0.028619           38.1470
    10000000     5000000          0.056588           38.1470          0.059252           76.2939

C:\Users\meloc\OneDrive\Desktop>g++ -std=c++17 -O3 Pract3_prog2.cpp -o Pract3_prog2

C:\Users\meloc\OneDrive\Desktop>.\Pract3_prog2

COMPARACION DE ALGORITMOS
================================================================================================================================
     n | Tradicional 2D (s) |  Mem (MB) | Tradicional 1D (s) |  Mem (MB) |  Strassen 2D (s) |  Mem (MB) |  Strassen 1D (s) |  Mem (MB)
--------------------------------------------------------------------------------------------------------------------------------
    16 |           0.000005 |     0.002 |           0.000002 |     0.002 |         0.000003 |     0.002 |         0.000002 |     0.002
    32 |           0.000017 |     0.009 |           0.000013 |     0.008 |         0.000057 |     0.053 |         0.000030 |     0.045
    64 |           0.000123 |     0.033 |           0.000100 |     0.031 |         0.000402 |     0.195 |         0.000134 |     0.180
   128 |           0.001058 |     0.128 |           0.002001 |     0.125 |         0.003073 |     0.750 |         0.001061 |     0.719
   256 |           0.009525 |     0.506 |           0.021452 |     0.500 |         0.019447 |     2.937 |         0.008084 |     2.875
   512 |           0.087716 |     2.012 |           0.503919 |     2.000 |         0.126018 |    11.623 |         0.044857 |    11.500
================================================================================================================================
Todas las multiplicaciones produjeron resultados iguales.
```

```
================================================================================
QUICKSELECT - TIEMPO Y CONSUMO DE MEMORIA
================================================================================
         n          k      Tiempo 32(s)    Memoria 32(MB)     Tiempo 64(s)   Memoria 64(MB)
--------------------------------------------------------------------------------
      1000        500          0.000432            0.0038         0.000320           0.0076
      5000       2500          0.001919            0.0191         0.001766           0.0381
     10000       5000          0.004947            0.0381         0.004111           0.0763
     50000      25000          0.014967            0.1387         0.004216           0.3715
    100000      50000          0.027753            0.3815         0.029502           0.7629
    500000     250000          0.189969            1.9073         0.199519           3.8147
   1000000     500000          0.325596            3.8147         0.350676           7.6294
   5000000    2500000          1.571735           19.0735         1.623334          38.1470
  10000000    5000000          2.154476           38.1470         2.231174          76.2939

C:\Users\meloc\OneDrive\Desktop>python Pract3_programa2.py

COMPARACION DE ALGORITMOS
================================================================================================================
     n | Tradicional 2D (s) | Mem (MB) | Tradicional (s) | Mem (MB) | Strassen 2D (s) | Mem (MB) | Strassen 1D (s) | Mem (MB)
----------------------------------------------------------------------------------------------------------------
    16 |          0.009712 |    0.002 |       0.009794 |    0.002 |       0.009723 |    0.002 |        0.009883 |    0.002
    32 |          0.076016 |    0.008 |       0.075713 |    0.008 |       0.068789 |    0.032 |        0.068839 |    0.032
    64 |          0.614079 |    0.031 |       0.618150 |    0.031 |       0.490335 |    0.120 |        0.487254 |    0.120
   128 |          5.074541 |    0.125 |       5.094543 |    0.125 |       3.581939 |    0.471 |        3.567903 |    0.471
   256 |         40.919174 |    0.500 |      53.153677 |    0.500 |      35.694036 |    1.877 |       40.006469 |    1.877
   512 |        327.353482 |    2.012 |     432.128743 |    2.000 |     255.483921 |    7.508 |      284.152893 |    7.480
================================================================================================================
Todas las multiplicaciones produjeron resultados iguales.

C:\Users\meloc\OneDrive\Desktop>
```

11. Conclusiones

La práctica permitió comprobar experimentalmente que la eficiencia real de un algoritmo no depende únicamente de su complejidad teórica. Quickselect mostró un comportamiento eficiente, pero la implementación en C++ obtuvo tiempos considerablemente menores que Python.

Para la multiplicación de matrices, Strassen presenta una ventaja teórica sobre el método tradicional, pero su beneficio depende del tamaño de las matrices y de la representación utilizada. En C++ se observó que Strassen 2D no superó al método tradicional 2D para n=512, mientras que Strassen 1D obtuvo el mejor tiempo de las cuatro variantes.

La diferencia más importante entre lenguajes aparece en los tiempos de ejecución. C++ aprovecha la compilación a código nativo, tipos numéricos de bajo nivel y optimizaciones del compilador. Python, aunque ofrece ventajas de simplicidad y productividad, presenta un costo mayor cuando se ejecutan directamente millones de operaciones dentro de bucles y estructuras recursivas. Por ello, para las implementaciones evaluadas y con los tamaños utilizados, C++ resulta claramente más eficiente en tiempo.

Repositorio de la práctica: https://github.com/leonardogmc/Practica3-DaC